UTS PENGKODEAN Kelas G

FAKULTAS EKONOMIKA DAN BISNIS UNIVERSITAS DIPONEGORO

Jl. Prof. Moeljono S. Trastotenojo, Kec. Tembalang, Kota Semarang 50275

Telp. (024) 76486841



Disusun Oleh:

Dandy Rizkiansah (12030124130135)

SOAL 1

1.1 Perancangan skema relasional (ERD)

Skema menggunakan tiga tabel. chart_of_accounts menyimpan master akun; journal_headers menyimpan identitas dokumen jurnal; journal_lines menyimpan baris debit/kredit yang terkait dengan satu header dan satu akun.

```
chart_of_accounts
- account_id (PK)
- account_code (UNIQUE)
- account_name
- normal_balance
- report_class

journal_headers
- header_id (PK)
- trx_date
- reference_no (UNIQUE)
- description
- posting_status

journal_lines
- line_id (PK)
- header_id (FK -> journal_headers.header_id)
- account_id (FK -> chart_of_accounts.account_id)
- debit
- credit
```

Relasi: chart_of_accounts 1 — N journal_lines; journal_headers 1 — N journal_lines. Setiap baris jurnal harus memiliki satu akun valid dan satu dokumen induk. CHECK constraint pada tiap baris menjaga nominal tidak negatif dan tidak mengizinkan debit serta kredit sama-sama nol. Keseimbangan total debit dan kredit perlu divalidasi pada tingkat transaksi/header saat posting karena CHECK biasa tidak dapat membandingkan agregat banyak baris.

1.2 DDL PostgreSQL

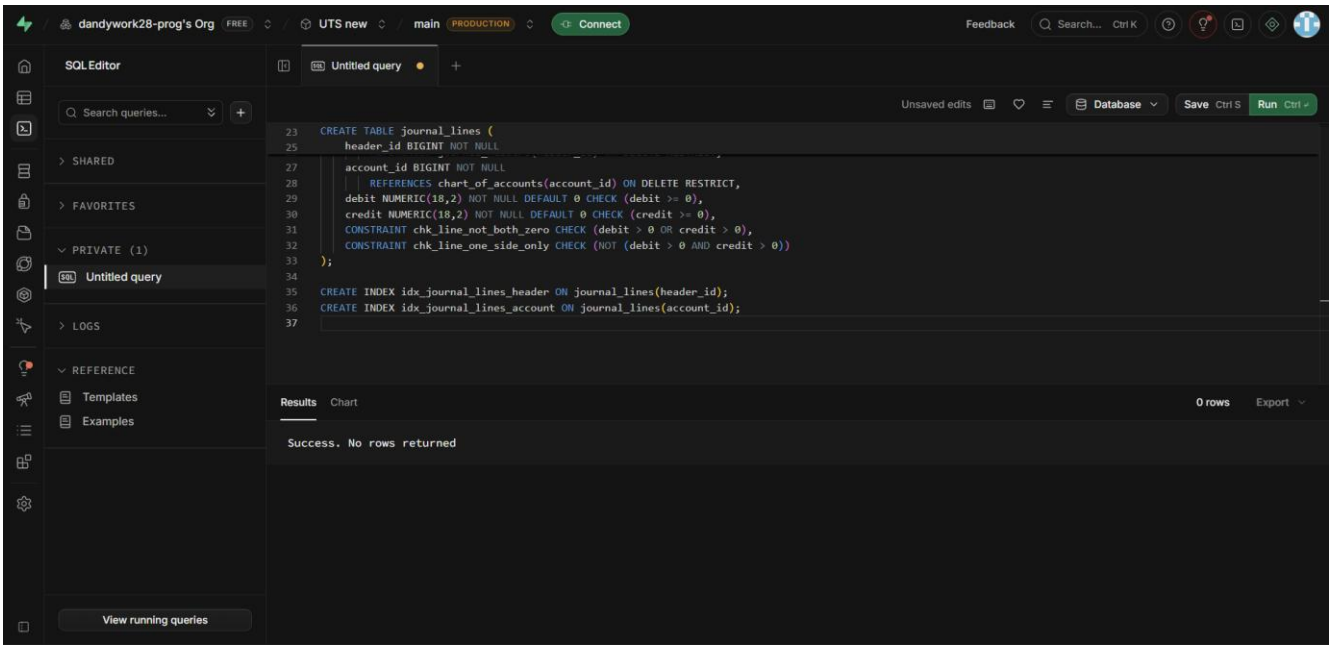
```
CREATE TABLE chart_of_accounts (
    account_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    account_code VARCHAR(20) NOT NULL UNIQUE,
    account_name VARCHAR(150) NOT NULL,
    normal_balance VARCHAR(6) NOT NULL
        CHECK (normal_balance IN ('DEBIT', 'CREDIT')),
    report_class VARCHAR(30) NOT NULL
        CHECK (report_class IN
          ('ASSET','LIABILITY','EQUITY','REVENUE','EXPENSE'))
);

CREATE TABLE journal_headers (
    header_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    trx_date DATE NOT NULL,
    reference_no VARCHAR(50) NOT NULL UNIQUE,
    description TEXT NOT NULL,
    posting_status VARCHAR(10) NOT NULL DEFAULT 'DRAFT'
        CHECK (posting_status IN ('DRAFT','POSTED','REVERSED')),
    created_by TEXT NOT NULL DEFAULT current_user,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE journal_lines (
    line_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    header_id BIGINT NOT NULL
        REFERENCES journal_headers(header_id) ON DELETE RESTRICT,
    account_id BIGINT NOT NULL
        REFERENCES chart_of_accounts(account_id) ON DELETE RESTRICT,
    debit NUMERIC(18,2) NOT NULL DEFAULT 0 CHECK (debit >= 0),
    credit NUMERIC(18,2) NOT NULL DEFAULT 0 CHECK (credit >= 0),
    CONSTRAINT chk_line_not_both_zero CHECK (debit > 0 OR credit > 0),
    CONSTRAINT chk_line_one_side_only CHECK (NOT (debit > 0 AND credit > 0))
);

CREATE INDEX idx_journal_lines_header ON journal_lines(header_id);
CREATE INDEX idx_journal_lines_account ON journal_lines(account_id);
```



Catatan integritas: constraint chk_line_one_side_only menetapkan satu baris hanya berada di sisi debit atau kredit. Untuk memastikan total debit = total kredit sebelum status menjadi POSTED, aplikasi/backend harus menjalankan validasi agregat dalam satu transaksi database (atau menggunakan trigger constraint/deferred yang dirancang khusus). Jangan mengandalkan validasi frontend saja karena API dapat dipanggil langsung.

1.3 Analisis audit trail dan 3NF

Pemisahan Header-Line menghubungkan setiap detail nominal dengan dokumen induk yang menyimpan tanggal, nomor bukti, deskripsi, dan status posting. Auditor dapat menelusuri transaksi dari dokumen ke akun-akun yang terdampak, memeriksa perubahan status, dan menguji rincian tanpa kehilangan konteks transaksi. Foreign key dan ON DELETE RESTRICT mencegah penghapusan header/akun yang masih memiliki rincian historis.

Model terpisah lebih sesuai dengan 3NF: atribut akun seperti kode, nama, tipe saldo normal, dan klasifikasi laporan disimpan sekali pada chart_of_accounts; atribut dokumen disimpan pada journal_headers; nominal dan referensi akun disimpan pada journal_lines. Model flat akan mengulang atribut akun dan dokumen pada setiap baris, menimbulkan redundansi serta anomali pembaruan, penyisipan, dan penghapusan.

SOAL 2

2.1 Trigger pembaruan stok dan pencegahan stok negatif

Asumsi: PURCHASE menambah stok, SALE mengurangi stok, dan ADJUSTMENT menggunakan qty bertanda positif/negatif untuk menaikkan/menurunkan stok. Trigger mengunci baris item dengan UPDATE atomik. Karena transaksi yang gagal akan me-rollback statement, pencatatan mutasi dan perubahan stok tidak terpisah.

```
-- Tabel sesuai struktur pada soal
CREATE OR REPLACE FUNCTION trg_update_stock_and_journal()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_delta INTEGER;
    v_new_stock INTEGER;
BEGIN
    IF NEW.qty IS NULL OR NEW.qty = 0 THEN
        RAISE EXCEPTION 'Qty tidak boleh NULL atau nol';
    END IF;

    IF NEW.trx_type = 'PURCHASE' THEN
        IF NEW.qty < 0 THEN
            RAISE EXCEPTION 'Qty PURCHASE harus positif';
        END IF;
        v_delta := NEW.qty;
    ELSIF NEW.trx_type = 'SALE' THEN
        IF NEW.qty < 0 THEN
            RAISE EXCEPTION 'Qty SALE harus positif';
        END IF;
        v_delta := -NEW.qty;
    ELSIF NEW.trx_type = 'ADJUSTMENT' THEN
        v_delta := NEW.qty;
    ELSE
        RAISE EXCEPTION 'Tipe transaksi tidak dikenal: %', NEW.trx_type;
    END IF;

    -- UPDATE mengunci baris item; transaksi bersamaan akan menunggu.
    UPDATE inventory_items
       SET stock_qty = stock_qty + v_delta
     WHERE item_id = NEW.item_id
       AND stock_qty + v_delta >= 0
     RETURNING stock_qty INTO v_new_stock;

    IF NOT FOUND THEN
        RAISE EXCEPTION
          'Item % tidak ditemukan atau stok tidak mencukupi untuk transaksi %',
          NEW.item_id, NEW.trx_type;
    END IF;

    -- Titik integrasi jurnal: jurnal Persediaan/HPP dapat dibuat di sini
    -- atau melalui prosedur posting yang menerima biaya unit dan akun terkait.
    RETURN NEW;
END;
$$;

CREATE TRIGGER inventory_transaction_after_insert
AFTER INSERT ON inventory_transactions
FOR EACH ROW
EXECUTE FUNCTION trg_update_stock_and_journal();
```

Penjelasan: PURCHASE menambah stock_qty, SALE menguranginya, dan ADJUSTMENT mengubahnya sesuai tanda qty. Untuk SALE, klausa WHERE stock_qty + v_delta >= 0 membuat pengurangan gagal bila stok tidak cukup; RAISE EXCEPTION membatalkan INSERT transaksi dan perubahan stok. Nama fungsi mengikuti instruksi soal. Meski namanya mencakup “journal”, contoh ini belum mem-posting jurnal GL karena struktur soal belum menyediakan akun Persediaan/HPP, harga pokok transaksi, atau tabel jurnal yang terhubung. Untuk implementasi penuh, tentukan akun dan nilai biaya transaksi lalu buat debit/kredit dalam transaksi database yang sama.

2.2 Concurrency / race condition

Dua kasir dapat membaca stok 1 secara bersamaan lalu sama-sama menganggap penjualan dapat dilakukan. Gunakan transaksi eksplisit dan SELECT ... FOR UPDATE untuk mengunci baris item sebelum mengecek stok dan memasukkan transaksi. Kasir kedua menunggu; setelah transaksi pertama commit, ia membaca stok terbaru (0) dan ditolak. Trigger di atas juga memakai UPDATE atomik sehingga menjadi lapisan pertahanan tambahan.

```
BEGIN;

SELECT stock_qty
FROM inventory_items
WHERE item_id = 'ITM-001'
FOR UPDATE;

-- Aplikasi/backend memeriksa stock_qty >= jumlah yang akan dijual.
-- Jika stok cukup:
INSERT INTO inventory_transactions (item_id, trx_type, qty)
VALUES ('ITM-001', 'SALE', 1);

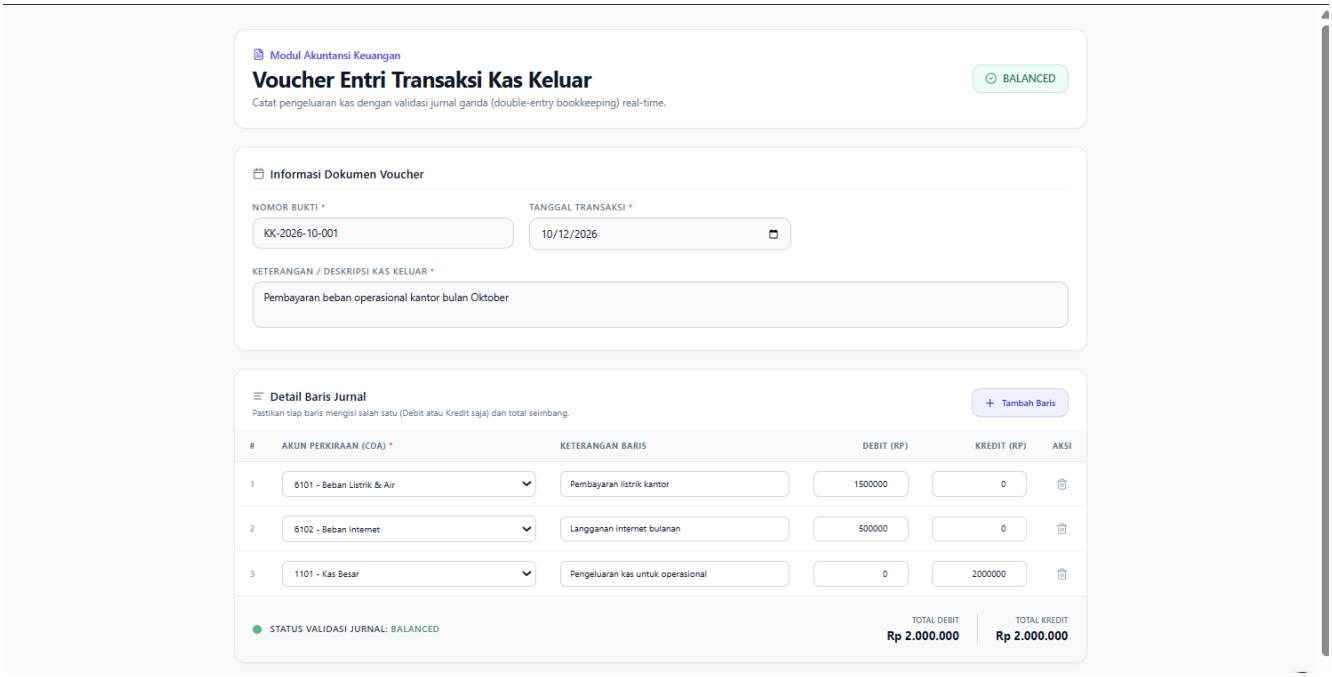
COMMIT;
```

Jika stok tidak cukup, backend harus ROLLBACK dan mengembalikan pesan gagal. Alternatifnya adalah isolation level SERIALIZABLE dengan penanganan retry ketika PostgreSQL mengembalikan serialization failure. FOR UPDATE biasanya lebih sederhana untuk mengunci satu item tertentu.

SOAL 3

3.1(a) Rancangan UI dan prompt Gemini Canvas

Rancangan UI memuat nomor bukti, tanggal, deskripsi, serta tabel baris jurnal dinamis (akun, keterangan, debit, kredit). Total debit dan kredit dihitung setiap kali input berubah. Status “BALANCED” hanya tampil bila total debit sama dengan total kredit dan lebih besar dari nol; tombol Submit aktif hanya jika syarat tersebut terpenuhi dan setiap baris valid.



Ditulis dengan prompt

“Buat purwarupa web responsif untuk “Voucher Entri Transaksi Kas Keluar” menggunakan React dan Tailwind CSS. Sediakan field nomor bukti, tanggal, deskripsi, dan tabel baris jurnal yang bisa ditambah/dihapus dengan kolom akun, keterangan, debit, kredit. Hitung total debit, total kredit, dan selisih secara real-time. Validasi agar nominal tidak negatif dan tiap baris hanya mengisi debit atau kredit. Tampilkan indikator BALANCED jika total debit = total kredit dan totalnya lebih besar dari nol; selain itu tampilkan NOT BALANCED beserta selisih. Nonaktifkan Submit bila tidak seimbang, total nol, field wajib kosong, atau tidak ada baris. Saat submit, kirim payload ke backend, jangan hanya mengandalkan validasi browser. Tampilkan loading, error, dan sukses. Buat kode mudah dibaca dan beri komentar untuk logika validasi. Jelaskan bahwa backend tetap harus memvalidasi ulang debit-kredit dan status posting di database.”

- Perancangan purwarupa modul Voucher Entri Transaksi Kas Keluar berbasis web dilakukan dengan memanfaatkan kerangka kerja interaktif. Formulir dibuat secara dinamis dengan dilengkapi fitur *live balanced indicator* yang menampilkan perhitungan total debit dan kredit secara real-time. Selain itu, diterapkan validasi *zero discrepancy* untuk memastikan tombol Submit hanya dapat digunakan ketika total debit dan kredit seimbang serta memiliki nilai lebih dari nol.

3.1(b) NotebookLM

Untuk transaksi kas keluar, akun Kas/Bank pada umumnya dikredit sebesar kas yang dibayarkan dan akun lawan didebit sesuai substansi (misalnya beban, utang, atau aset). Total debit dan kredit wajib sama. Validasi UI membantu mencegah kesalahan input, tetapi bukan bukti kepatuhan PSAK secara menyeluruh. Pengakuan dan klasifikasi kas/setara kas harus ditinjau berdasarkan sumber PSAK yang benar-benar tersedia; status setara kas bergantung pada karakteristik instrumen dan tidak boleh ditentukan hanya dari nama akun.

NotebookLM dimanfaatkan sebagai asisten analitis berbasis pengetahuan (*knowledge base*) untuk mengevaluasi logika bisnis dan meninjau kesesuaian sistem dengan Pernyataan Standar Akuntansi Keuangan

(PSAK) serta standar pelaporan keuangan terkait pengakuan kas dan setara kas. Proses ini mencakup penelaahan terhadap mekanisme pencatatan, pemisahan fungsi, dan pengendalian transaksi berdasarkan kerangka konseptual akuntansi keuangan serta prinsip pengendalian internal yang relevan.

3.2 Backend, immutability, dan audit trail

Aturan immutability harus ditegakkan di database, bukan hanya pada frontend. Setelah status POSTED, UPDATE atau DELETE ditolak. Koreksi dilakukan dengan jurnal pembalik/reversing entry baru yang merujuk transaksi asal, bukan mengedit histori. **Implementasi Proteksi Integritas dan Immutability Transaksi**

Proteksi integritas transaksi diterapkan pada tabel journal_headers dan journal_lines melalui mekanisme *database trigger*. Sebelum implementasi dilakukan, kolom hash_checksum ditambahkan pada tabel journal_headers untuk menyimpan nilai hash dari data transaksi. Trigger pada tabel header dan detail berfungsi mencegah perubahan maupun penghapusan transaksi yang telah berstatus POSTED. Apabila terdapat upaya untuk memodifikasi atau menghapus transaksi yang telah diposting, sistem akan menolak operasi tersebut dan menampilkan pesan kesalahan. Koreksi transaksi dilakukan melalui mekanisme *reversing entry*, bukan dengan mengubah catatan transaksi yang telah diposting.

Pada sisi backend, proses pencatatan transaksi diawali dengan autentikasi pengguna dan pengambilan identitas pengguna (*user ID*) dari sesi yang tepercaya, bukan dari *request body*. Selanjutnya, sistem memulai transaksi database dan memvalidasi kelengkapan header, validitas akun, nominal transaksi, serta keseimbangan total debit dan kredit dengan nilai total yang lebih besar dari nol. Setelah seluruh validasi terpenuhi, sistem menyimpan data header dan rincian jurnal, menghitung nilai hash berdasarkan representasi data penting yang telah diserialisasi secara konsisten, kemudian menyimpannya pada kolom hash_checksum. Status transaksi diubah menjadi POSTED setelah seluruh pemeriksaan berhasil, lalu transaksi database diakhiri dengan proses *commit*. Apabila terjadi kegagalan pada salah satu tahapan, sistem menjalankan *rollback* untuk membatalkan seluruh perubahan sehingga tidak terbentuk pencatatan yang tidak konsisten.

Kolom created_by digunakan untuk mencatat identitas pengguna yang membuat transaksi, sedangkan created_at menyimpan waktu pembuatan secara otomatis menggunakan tipe data *timestamp*. Adapun hash_checksum menyimpan *digest*, misalnya menggunakan algoritma SHA-256, yang dapat dibandingkan dengan hasil perhitungan ulang untuk membantu mendeteksi perubahan data. Namun, mekanisme hash tersebut tidak dapat menjamin integritas secara mandiri apabila pihak yang tidak berwenang juga dapat mengubah data dan nilai hash-nya. Oleh karena itu, penguatan keamanan dapat dilakukan melalui pembatasan hak akses database, penggunaan *audit log* yang bersifat *append-only*, serta penyimpanan hash pada sistem terpisah. Informasi sensitif, seperti kata sandi dan *secret*, tidak boleh dicatat dalam log.

Untuk mendukung penerapan pada lingkungan produksi, mekanisme proteksi perlu mencakup pencegahan perubahan status transaksi secara tidak sah, perubahan header_id pada rincian jurnal, serta akses melalui jalur *service-role* atau administrator. Selain itu, hak akses database harus diatur agar hanya pihak yang berwenang yang dapat melakukan proses *posting* maupun perubahan terhadap tabel terkait. Dengan demikian, mekanisme *trigger*, validasi backend, pencatatan audit, dan pengaturan hak akses bekerja secara terpadu untuk menjaga integritas serta konsistensi data transaksi.

Kesimpulan

Desain menggunakan pemisahan master-header-line, constraint relasional dan nominal, trigger stok atomik, penguncian baris untuk concurrency, validasi balance pada UI sekaligus backend, serta proteksi transaksi POSTED. Implementasi produksi tetap memerlukan pengujian, autentikasi/otorisasi, posting jurnal persediaan dengan akun dan biaya yang jelas, dan verifikasi sumber PSAK yang relevan.

Github dapat diakses pada : https://github.com/dandywork28-prog/UTS-KODE.git